Caso Práctico Unidad 3

Desarrollo de una Aplicación de Generación de Imágenes y Edición de Contenido utilizando Amazon Bedrock

1. Objetivos y alcance

Descripción general de la solución propuesta

Se ha desarrollado GenAI Marketing Studio, un prototipo de aplicación web orientado a una agencia de marketing y publicidad. La solución centraliza en una única interfaz dos tareas creativas principales: la generación de imágenes a partir de descripciones textuales y la transformación de contenido escrito. Además, incorpora elementos de flujo de trabajo como roles, historial de versiones, comentarios y un apartado de ética y seguridad.

La aplicación se ejecuta localmente mediante Streamlit y se conecta a Amazon Bedrock a través del SDK de AWS para Python (boto3). La generación de texto se ha configurado con Anthropic Claude Haiku 4.5 y la generación de imágenes con Stability AI Stable Diffusion 3.5 Large. La región seleccionada es us-west-2 (Oregón).

Objetivos funcionales y técnicos

- Permitir que un diseñador introduzca una descripción, seleccione un estilo y un formato, y solicite una imagen generada por IA.
- Permitir que un redactor resuma, amplíe, corrija o genere variaciones de un texto mediante Claude.
- Mantener un historial de versiones para comparar y restaurar contenido anterior.
- Representar un flujo colaborativo mediante los roles Diseñador, Redactor y Aprobador, junto con comentarios y notas.
- Aplicar controles básicos de moderación, privacidad, uso responsable y protección frente a intentos simples de prompt injection.
- Mantener la configuración sensible fuera del código entregado y utilizar credenciales AWS configuradas localmente.

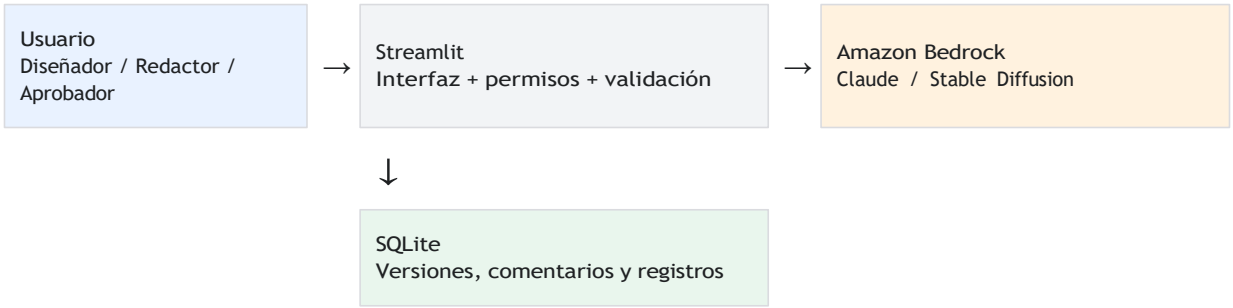
Tecnologías, plataformas y servicios seleccionados

Tecnología	Uso en el proyecto
Python 3	Lógica de la aplicación e integración con servicios.
Streamlit	Interfaz web del prototipo.
Amazon Bedrock	Acceso gestionado a los modelos generativos.
Claude Haiku 4.5	Resumen, ampliación, corrección y variaciones de texto.
Stable Diffusion 3.5 Large	Generación de imágenes desde prompts.
boto3	Comunicación entre Python y Amazon Bedrock.
SQLite	Persistencia local de versiones, comentarios e imágenes.

2. Diseño de la solución

Arquitectura general de la aplicación

La arquitectura se ha planteado de forma modular y sencilla para un prototipo académico. El usuario interactúa con la interfaz Streamlit. La aplicación valida la entrada y, según la acción seleccionada, utiliza boto3 para realizar una petición a Amazon Bedrock. Los resultados y metadatos se almacenan localmente en SQLite para alimentar el historial, la galería y los comentarios.



Módulos y componentes principales

- Generación de imágenes: Prompt, estilo (realismo, anime, pintura al óleo o minimalista), formato, llamada a Stable Diffusion, galería y descarga.
- Edición de contenido: Entrada de texto y cuatro operaciones: resumir, ampliar, corregir y crear variaciones mediante Claude.
- Historial: Registro de versiones, comparación entre resultados y restauración de versiones anteriores.
- Colaboración: Identificación de usuario, selección de rol, proyecto activo y comentarios/notas.
- Ética y seguridad: Moderación básica, reglas de uso, prevención simple de prompt injection y recomendaciones de seguridad para producción.

Diseño de roles, permisos y mecanismos de colaboración

Se han definido tres perfiles. El Diseñador se orienta a la generación de imágenes; el Redactor a la edición de contenido; y el Aprobador a la revisión de ambos tipos de activos. Los roles controlan qué acciones se muestran en la interfaz. Los comentarios se guardan asociados al usuario, rol, proyecto y fecha, facilitando la retroalimentación.

3. Desarrollo e implementación

Configuración de servicios y modelos de IA

La cuenta AWS se configuró para trabajar con Amazon Bedrock en us-west-2. Se verificó el acceso desde AWS CLI y desde Python. La aplicación utiliza un cliente bedrock-runtime creado con boto3. Para texto se configuró el perfil de inferencia de EE. UU. de Claude Haiku 4.5; para imagen se configuró Stable Diffusion 3.5 Large.

La edición de contenido utiliza la API Converse de Bedrock. Se han definido parámetros distintos según la tarea: temperaturas bajas para corrección y resumen, y valores más altos para ampliación y generación de variaciones. Esta decisión busca equilibrar consistencia y creatividad.

Operación	Temperatura	Top-P	Máx. tokens	Finalidad
Resumir	0,3	0,90	500	Respuesta estable y concisa.
Corregir	0,2	0,85	700	Priorizar precisión.
Ampliar	0,6	0,95	900	Mayor riqueza expresiva.
Variaciones	0,9	0,98	1000	Favorecer diversidad creativa.

Prompt de sistema y control de entrada

Claude recibe instrucciones de sistema orientadas al rol de redactor profesional de marketing. Se le solicita no inventar características del producto, no revelar instrucciones internas y evitar contenido ilegal, dañino, discriminatorio u ofensivo. Antes de enviar la entrada al modelo, la aplicación aplica un filtrado básico de términos y patrones asociados a intentos de anular las instrucciones del sistema.

Persistencia e implementación colaborativa

SQLite permite conservar el historial de textos, comentarios e información de las imágenes dentro del proyecto. Cada modificación puede quedar registrada como una versión independiente. Esto posibilita comparar resultados y recuperar una versión anterior sin depender de la memoria conversacional del modelo.

Las credenciales de AWS no se incluyen en el código ni en los archivos de entrega. El proyecto utiliza la cadena de credenciales estándar de AWS, configurada localmente mediante AWS CLI. El archivo .env.example se mantiene únicamente como referencia de configuración.

4. Validación de requisitos

Cumplimiento de los requisitos funcionales del caso

Requisito	Implementación realizada	Estado
Generación de imágenes	Prompt, estilos, formato, llamada a Stable Diffusion, galería y descarga.	Implementado
Edición con Claude	Resumen, ampliación, corrección y variaciones.	Implementado
Historial	Versiones, comparación y restauración.	Implementado
Roles y permisos	Diseñador, Redactor y Aprobador con acciones diferenciadas.	Prototipo funcional
Comentarios	Notas asociadas a usuario, rol y proyecto.	Implementado
Ética y seguridad	Moderación básica, privacidad, copyright, sesgos y prompt injection.	Implementado a nivel prototipo

Evidencia de la aplicación desarrollada

La siguiente captura muestra la interfaz ejecutándose en localhost. Se observan el selector de usuario, rol y proyecto, las pestañas funcionales, el formulario de generación de imágenes y la configuración de los modelos.

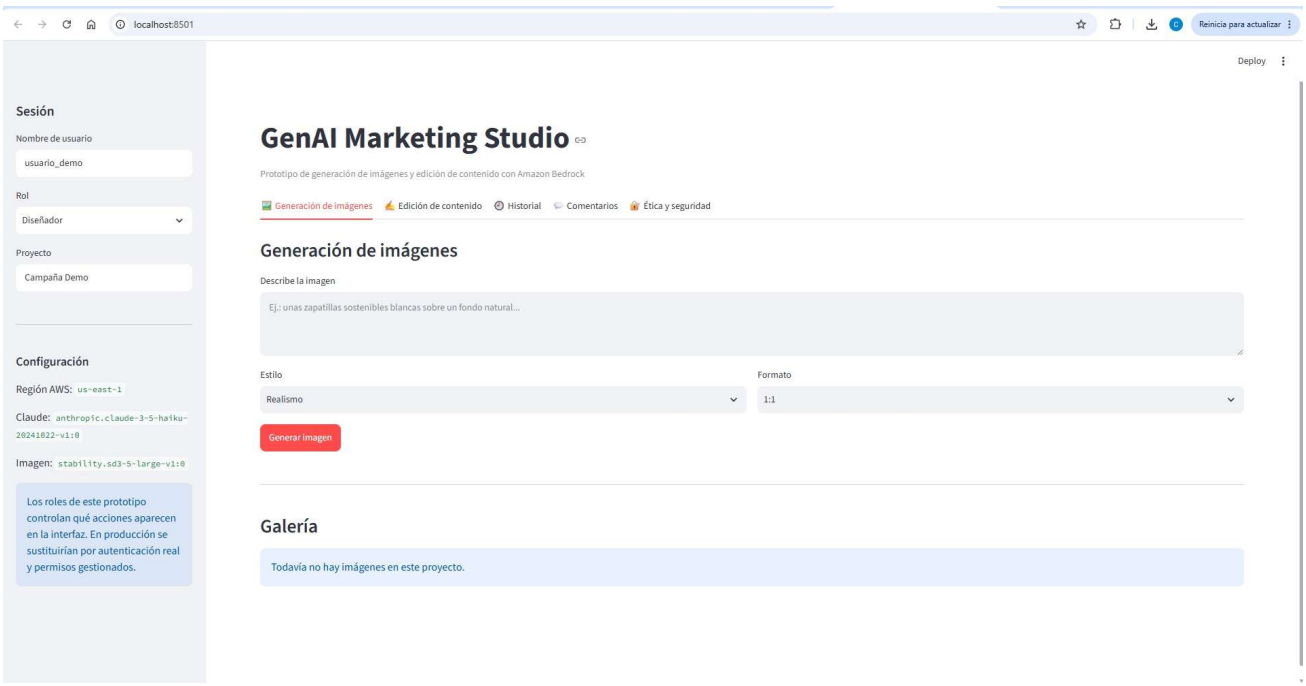


Figura 1. Interfaz del prototipo GenAI Marketing Studio.

Evidencias de generación o transformación mediante IA

Durante la validación se comprobó correctamente la autenticación de AWS CLI, el acceso al catálogo de modelos de Amazon Bedrock y la comunicación desde Python mediante boto3. Las llamadas alcanzaron los endpoints de inferencia de Claude y Stable Diffusion.

No obstante, al realizar las pruebas finales la cuenta AWS de nueva creación aplicó límites temporales de uso. Claude devolvió ThrottlingException: Too many tokens per day, mientras que Stable Diffusion devolvió ThrottlingException: Too many requests. Por este motivo no se incorporan resultados generados ficticios: la evidencia refleja de forma transparente el estado real de las pruebas.



Figura 2. Limitación temporal de cuota observada durante la prueba de Claude en Amazon Bedrock.

El error de throttling se produce después de que la petición alcance Amazon Bedrock. Por tanto, confirma que la autenticación, la región, el cliente boto3 y la identificación del modelo están siendo procesados por el servicio. La limitación corresponde a la cuota temporal de la cuenta, no a un resultado simulado.

Resultados obtenidos y comparación con los objetivos

Los objetivos de diseño e implementación se han alcanzado a nivel de prototipo: existe una interfaz integrada, los flujos de texto e imagen están conectados con Bedrock, se han desarrollado los mecanismos de historial, colaboración y seguridad, y el proyecto puede ejecutarse localmente. La principal desviación respecto al objetivo de validación completa es la imposibilidad temporal de obtener una salida final de los modelos por las cuotas de la cuenta AWS durante la sesión de pruebas.

Esta limitación no se oculta en la entrega y se contempla como una condición externa del entorno cloud. Una vez renovada o ampliada la cuota, las mismas funciones pueden volver a ejecutarse sin modificar la arquitectura del prototipo.

4. Entregables y conclusiones

Acceso a la aplicación funcional o prototipo

El prototipo se entrega junto con su código fuente y dependencias. La ejecución se realiza localmente desde la carpeta del proyecto con streamlit run app.py. Una vez iniciado, Streamlit habilita normalmente la dirección http://localhost:8501. Este enlace es local y requiere ejecutar previamente el proyecto; no se presenta como una URL pública desplegada.

Archivos principales de entrega: app.py, requirements.txt, README.md, .env.example y la base de datos local genai_marketing.db. Las credenciales AWS no se incluyen.

Principales hallazgos y lecciones aprendidas

- Amazon Bedrock permite integrar distintos proveedores de modelos desde un servicio común, pero cada modelo puede tener perfiles de inferencia, regiones y cuotas específicas.
- Separar interfaz, lógica de IA y persistencia facilita ampliar el prototipo sin rehacer la aplicación completa.
- El historial de la aplicación debe persistirse de forma independiente de la memoria conversacional del modelo.
- Las limitaciones de cuota y disponibilidad son parte del comportamiento real de una arquitectura cloud y deben contemplarse durante las pruebas.

Conclusiones

GenAI Marketing Studio demuestra cómo integrar IA generativa en un flujo de marketing que combina creación visual, edición de contenido y colaboración. El proyecto cubre los requisitos funcionales del caso mediante una solución sencilla y comprensible, adecuada para un prototipo académico.

La arquitectura propuesta permite evolucionar hacia un entorno productivo. Como mejoras futuras se plantea incorporar autenticación real, almacenamiento cloud, cifrado gestionado, permisos persistentes, Bedrock Guardrails, auditoría, despliegue público seguro y monitorización de costes y cuotas. También sería conveniente añadir pruebas automáticas y una evaluación sistemática de calidad para los resultados de texto e imagen.

5. Referencias y anexos

Fuentes utilizadas para fundamentar la propuesta

Instituto Europeo de Posgrado. Generative AI - Caso Práctico Unidad 3. Enunciado.

Instituto Europeo de Posgrado. Material docente de la Unidad 3: arquitectura, parámetros y construcción de aplicaciones de IA generativa.

Amazon Web Services. Amazon Bedrock User Guide. Documentación oficial de modelos, inferencia, cuotas y SDK. Amazon Web Services. AWS SDK for Python (Boto3) - Bedrock Runtime. Documentación oficial.

Anthropic. Documentación del modelo Claude disponible a través de Amazon Bedrock. Stability AI / Amazon Web Services. Documentación de Stable Diffusion 3.5 Large en Amazon Bedrock. Streamlit. Documentación oficial para desarrollo de aplicaciones de datos en Python.

Declaración de uso de herramientas de IA

Para la realización de esta práctica se ha utilizado ChatGPT como herramienta de apoyo en la estructuración del prototipo, revisión del código y resolución de errores de configuración. La configuración de la cuenta AWS, las pruebas de Amazon Bedrock y la ejecución del prototipo se han realizado sobre el entorno de trabajo utilizado para la práctica.

Anexo A. Estructura del proyecto

Archivo	Finalidad
app.py	Aplicación principal Streamlit e integración con Bedrock.
requirements.txt	Dependencias necesarias para ejecutar el proyecto.
README.md	Instrucciones de instalación, configuración y pruebas.
.env.example	Ejemplo de variables configurables sin credenciales reales.
genai_marketing.db	Persistencia SQLite del prototipo.

Anexo B. Ejecución básica

1. Instalar las dependencias con pip install -r requirements.txt.
2. Configurar credenciales AWS mediante AWS CLI y utilizar la región us-west-2.
3. Ejecutar streamlit run app.py.
4. Abrir la dirección local indicada por Streamlit y acceder a los módulos del prototipo.